```
// MOVE PWM WIRE TO PIN 9

#include <Encoder.h>
#include <math.h>
#define ENCODER_OPTIMIZE_INTERRUPTS

// Pin Declarations
const int MotIn1 = 4;
const int MotIn2 = 5;
const int PWMout = 9;

const int encPinA = 2;
const int encPinB = 3;

Encoder encoder(encPinB, encPinA);

// Encoder + geometry
float encoderResolution = 48;
float pos = 0;

float rh = 0.085;
float rp = 0.005;
float rs = 0.075;

// Kinematics
volatile float xh = 0;
volatile float xh_prev = 0;

volatile float tPulley;
volatile float tS;

// Velocity filter
float vh1 = 0;
float vh1_prev = 0;

float dt = 0.001;
float fc = 40;
float r = exp(-2*M_PI*fc*dt);
```

```
float duty;

// ----------- MODES -----------
enum hModes {
  HARD_STOP_VIBRATE,
  PIANO_KEY,
  PUSH_LEFT,
  PUSH_RIGHT
};

volatile hModes hapticMode = HARD_STOP_VIBRATE;

// ---------- SETUP ----------
void setup() {

  // PWM setup (20kHz)
  TCCR1A = (1 << COM1A1);
  TCCR1B = (1 << WGM13) | (1 << CS10);
  ICR1 = 400;

  pinMode(MotIn1, OUTPUT);
  pinMode(MotIn2, OUTPUT);
  pinMode(PWMout, OUTPUT);

  digitalWrite(MotIn1, HIGH);
  digitalWrite(MotIn2, LOW);
  OCR1A = 0;

  // Timer2 → 1kHz loop
  cli();

  TCCR2A = 0;
  TCCR2B = 0;

  TCCR2A |= (1 << WGM21);
  TCCR2B |= (1 << CS22);

  OCR2A = 249;
  TIMSK2 |= (1 << OCIE2A);
```

```
  sei();

  Serial.begin(115200);
  Serial.println("Send '1' for Hard Stop Vibrate, '2' for Piano Key");
}

// ---------- MOTOR ----------
void setMotor(int motCommand) {

  if (motCommand > 400) motCommand = 400;
  if (motCommand < -400) motCommand = -400;

  if (motCommand > 0) {
    digitalWrite(MotIn1, HIGH);
    digitalWrite(MotIn2, LOW);
    OCR1A = motCommand;
  } else {
    digitalWrite(MotIn1, LOW);
    digitalWrite(MotIn2, HIGH);
    OCR1A = -motCommand;
  }
}

// ---------- HAPTIC FUNCTION 1: HARD STOP + VIBRATE ----------
float hardStopVibrate(float x, float v) {

  float x_limit = 2.0;
  float k_wall  = 300;
  float b       = 10;
  float k_return = 50;
  float vib_amp  = 80;
  float vib_freq = 150;

  static float t = 0;
  t += dt;

  if (x >= x_limit) {
    return -k_wall * (x - x_limit)
           - b * v
           + vib_amp * sin(2 * M_PI * vib_freq * t);
```

```
  } else {
    return -k_return * x - b * v;
  }
}

// ---------- HAPTIC FUNCTION 2: PIANO KEY ----------
float pianoKey(float x, float v) {

  float x_bottom   = 8.0;   // 🔴 TUNE: push this way out so you have real
travel
  float x_top      = 0.0;   // negative direction hard wall
  float k_hard     = 150;   // wall stiffness (both ends)
  float k_soft     = 15;    // resistance during travel — keep this low
  float k_return   = 200;   // 🔴 strong return spring across entire
travel range
  float b_press    = 3;     // light damping while pressing
  float b_return   = 1;     // even lighter damping on return so spring
wins

  if (x >= x_bottom) {
    // BOTTOMED OUT
    return -k_hard * (x - x_bottom) - b_press * v;
  }
  else if (x <= x_top) {
    // NEGATIVE WALL
    return -k_hard * (x - x_top) - b_press * v;
  }
  else if (v < 0) {
    // ACTIVELY PRESSING DOWN — soft felt resistance
    return -k_soft * x - b_press * v;
  }
  else {
    // RELEASING — strong spring return, low damping
    return -k_return * x - b_return * v;
  }
}

// ---------- HAPTIC FUNCTION 3: PUSH LEFT ----------
float pushLeft() {
  float x_limit = -6.0;  // 🔴 TUNE: your left physical limit
```

```
  float k_push  = 150;    // base push strength
  float b       = 20;     // damping near the wall

  if (xh <= x_limit) {
    // at the wall — push back and damp hard
    return -k_push * (xh - x_limit) - b * vh1;
  } else {
    // taper force as it approaches the limit
    float proximity = 1.0 - constrain((x_limit - xh) / x_limit, 0.0, 1.0);
    return -k_push * proximity - b * vh1 * proximity;
  }
}
// ------------ PUSH RIGHT
float pushRight() {
  float x_limit = 6.0;    // 🔴 TUNE: your right physical limit
  float k_push  = 150;
  float b       = 20;

  if (xh >= x_limit) {
    // at the wall — push back and damp hard
    return -k_push * (xh - x_limit) - b * vh1;
  } else {
    float proximity = constrain(xh / x_limit, 0.0, 1.0);
    return k_push * (1.0 - proximity * proximity) - b * vh1 * proximity;
  }
}

// ---------- LOOP ----------
void loop() {

  // Check serial for mode switch
  if (Serial.available() > 0) {
    char input = Serial.read();

    if (input == '1') {
      hapticMode = HARD_STOP_VIBRATE;
      Serial.println("Mode: Hard Stop Vibrate");
    }
    else if (input == '2') {
      hapticMode = PIANO_KEY;
```

```
      Serial.println("Mode: Piano Key");

    }
    else if (input == '3') {
      hapticMode = PUSH_LEFT;
      Serial.println("Mode: Push Left");
    }
    else if (input == '4') {
      hapticMode = PUSH_RIGHT;
      Serial.println("Mode: Push Right");
    }
  }

  Serial.println(duty);
}

// ---------- ISR ----------
ISR(TIMER2_COMPA_vect) {

  pos = encoder.read();

  tPulley = (pos / encoderResolution) * 2 * M_PI;
  tS = (tPulley * rp) / rs;

  xh = 100 * (rh * rp * tPulley) / rs;

  vh1 = r * vh1_prev + (1 - r) * (xh - xh_prev) / dt;

  xh_prev = xh;
  vh1_prev = vh1;

switch (hapticMode) {
  case HARD_STOP_VIBRATE:
    duty = hardStopVibrate(xh, vh1);
    break;
  case PIANO_KEY:
    duty = pianoKey(xh, vh1);
    break;
  case PUSH_LEFT:
    duty = pushLeft();
```

```
    break;
  case PUSH_RIGHT:
    duty = pushRight();
    break;
}

  setMotor(duty);
}
```